AKENTEN APPIAH-MENKA UNIVERSITY
of Skills Training *and* Entrepreneurial Development

Hello!!!

Silence Hour

Use this time to revise your notes before we begin.

AKENTEN APPIAH-MENKA UNIVERSITY
of Skills Training *and* Entrepreneurial Development

ICT233

Web Technologies and Design

Lecture Eight

Today's Lecture

1.0
CSS Units

2.0
CSS Positioning

3.0
CSS Specificity

4.0
CSS Website Layout

Lecture Objectives

- By the end of the lecture, the student will be able to:
  - Explain CSS Units.
  - Describe the CSS positioning methods.
  - Determine the hierarchy of applying CSS rules to HTML elements
  - Create website layouts using CSS positioning methods.

CSS Units -- Overview

- CSS units are used to express measurements and lengths for various properties such as width, margin, and font-size.
- CSS units are categorized into **Absolute** and **Relative** units.
- The choice of unit is crucial for creating responsive and accessible web designs.
- Measurements and units are expressed as numeric values followed by a CSS unit such as **px**, **em**, **rem**, **vh** etc.
- **Note:** A whitespace cannot appear between the numeric value and the unit.
- However, if the value is 0, the unit can be omitted.

CSS Units -- Absolute Units

- Absolute units are fixed and do not change size relative to the parent element or the browser's viewport.
- Absolute units do not change when the screen size changes, and are not recommended for websites.
- They are best used when the physical output medium is known (like print) or for elements that require exact, non-scaling dimensions.

CSS Units -- Absolute Units

Unit	Description
px	pixels (1px = 1/96th of 1in)
cm	centimeters
mm	millimeters
in	inches (1in = 96px = 2.54cm)
pt	points (1pt = 1/72 of 1in)
pc	picas (1pc = 12 pt)

CSS Units -- Relative Units

- Relative units are used to specify a length relative to another length such as **parent element**, **root element**, or **viewport**.
- Relative length units scale better between different screen sizes, making them ideal for responsive web design.

CSS Units -- Relative Units

Unit	Description
em	Relative to the font-size of the parent element
rem	Relative to the font-size of the root HTML element
vw	Relative to 1% of the width of the viewport*. 100vw = full width of the viewport
vh	Relative to 1% of the height of the viewport*. 100vh = full height of the viewport
fr	A fractional unit. 1fr equals 1 part of the available space
%	Relative to the size of the parent element

CSS Positioning

- CSS positioning is about controlling the placement of elements within a web page.
- With CSS positioning, you can override the normal document flow.
- The position property specifies the positioning type for an element, can be set to one of the following values:
  - **static** - This is default
  - **relative**
  - **fixed**
  - **absolute**
  - **sticky**
- Elements are then positioned to their final location with the top, bottom, left, and right properties.

CSS position: static;

- All HTML elements are positioned static by default.
- Static positioned elements are not affected by the **top**, **bottom**, **left**, and **right** properties.
- An element with **position: static**; is always positioned according to the normal flow of the page.

```
div.static {
  position: static;
  border: 3px solid #73AD23;
}
```


CSS position: relative;

- An element with **position: relative**; is positioned relative to its normal position in the document flow.
- Setting the top, right, bottom, and left properties will cause the element to be adjusted away from its normal position.
- Other content will not be adjusted to fit into any gap left by the element.

```
div.relative {
        position: relative;
        left: 45px;
        border: 3px solid #73AD23;
}
```

CSS position: fixed;

- An element with **position: fixed**; is positioned relative to the viewport, which means it always stays in the same place even if the page is scrolled.
- The top, right, bottom, and left properties are used to set the final location of the element.
- A fixed element does not leave a gap in the page where it would normally have been located.

```
div.fixed {
        position: fixed;
        bottom: 0;
        right:0
        width: 275px;
        border: 3px solid #73AD23;
}
```


CSS position: absolute;

- An element with **position: absolute;** is positioned relative to the nearest positioned ancestor (with position other than static).
- However; if an absolute positioned element has no positioned ancestors, it uses the document body, and moves along with page scrolling.
- **Note:** Absolute positioned elements are removed from the normal document flow, and can overlap other elements.

```
div.relative {
 position: relative;   width: 400px;   height: 200px;
border: 3px solid green;
}
div.absolute {
 position: absolute; top: 80px;   right: 0;  width: 200px;
 height: 100px;   border: 3px solid red;
}
```

Position: sticky;

- An element with **position: sticky;** toggles between a relative and fixed position, depending on the scroll position.
- A sticky element is positioned relative until a certain scroll position is reached - then it "sticks" in that place (like position:fixed).
- **Note:** You must specify at least one of the top, right, bottom or left properties, for sticky positioning to work.

```
div.sticky {
        position: sticky;
        background-color: green;
        top: 2px;
        border: 2px solid #5CAF52;
}
```


All CSS Positioning Properties

Property	Description
position	Specifies the type of positioning for an element
top	Sets the top margin edge for a positioned box
right	Sets the right margin edge for a positioned box
bottom	Sets the bottom margin edge for a positioned box
left	Sets the left margin edge for a positioned box

CSS Specificity

- CSS Specificity is the rule that determines which CSS style is applied when multiple rules target the same element.
- If two or more CSS rules point to the same element, the declaration with the highest specificity will be applied to the HTML element.
- Consider the following code snippets

```
p { color: blue; }
.text { color: red; }
#main { color: green; }
```

```
<p id="main" class="text">Sample Paragraph</p>
```

Which CSS rule wins?

CSS Specificity – The Need

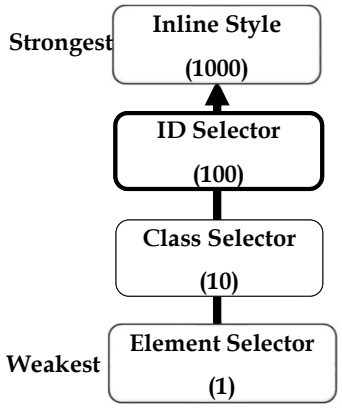
- Consider the following code snippets

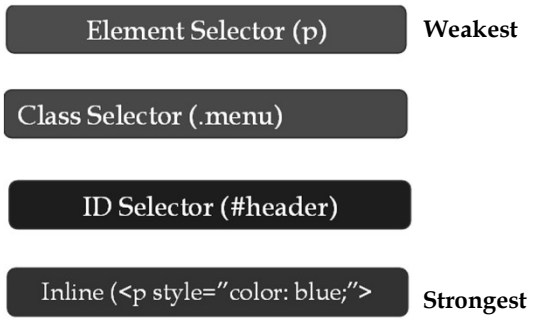
```
p { color: blue; }
p { color: red; }
p { color: green; }
```

- Without specificity, browsers would not know which instruction to follow.
- Specificity helps browsers resolve conflicts between CSS rules.
- Remember CSS is declarative and the possibility of multiple rules existing simultaneously is very high.

CSS Specificity -- Hierarchy

- Knowing the specificity hierarchy of selectors is a necessary skill for writing cleaner and unambiguous CSS rules.





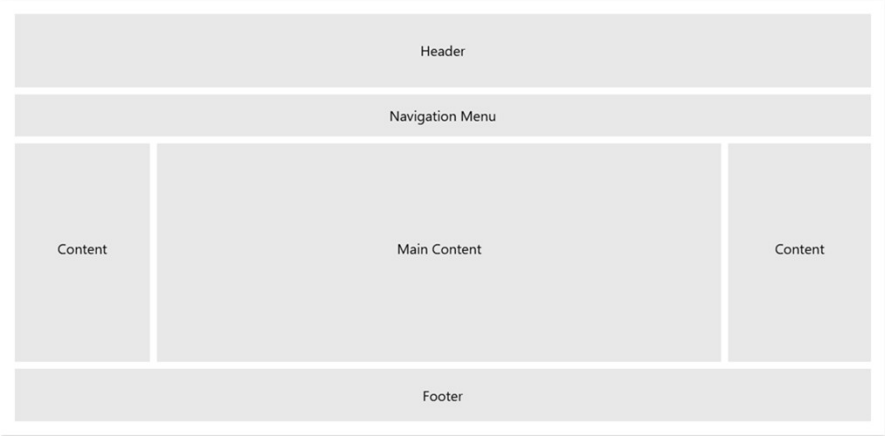
Web Technologies Lecture Slides by: Maxwell Dorgbefu Jnr.

CSS Website Layout Design

- This is the arrangement of **header, navigation, main content, sidebar,** and **footer** elements of websites
- It determines visual structure and flow of a website
- Website layout design uses the following CSS techniques:
  - Box Model
  - Flexbox*
  - Grid*
  - Positioning

CSS Website Layout Design

There are different layout designs to choose from for your website projects.
However, the structure below, is one of the most common.



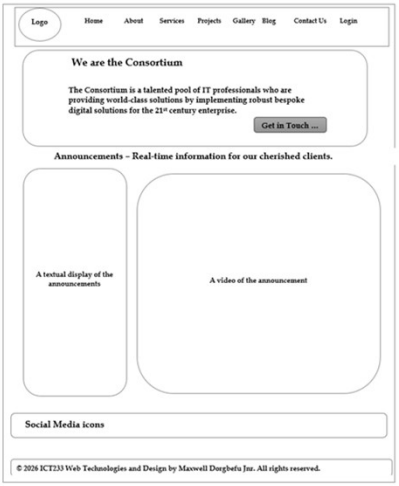
CSS Website Layout Design – Sample Codes

```
header {
  background-color: #f1f1f1;
  text-align: center;
  padding: 10px;
}
```

```
/* Style the topnav */
ul.topnav {
  display: flex;
  list-style-type: none;
  margin: 0;
  padding: 0;
  background-color: #333333;
}
/* Style links in topnav */
ul.topnav li a {
  display: block;
  color: #f1f1f1;
  padding: 14px 16px;
  text-decoration: none;
}
/* Change color on hover */
ul.topnav li a:hover {
  background-color: #dddddd;
  color: black;
}
```

Activity -- Layout Design

- We are switching to our code consoles now to design the layout shown in the diagram below



Our Next Lecture …

1. The basis of JavaScript
2. Writing and debugging JavaScript codes
3. Arrays and Functions in JavaScript